Exp No: 27

BINARY SEMAPHORE IN SYSTEMC

AIM:

To implement and simulate a Binary Semaphore using SystemC for task synchronization.

SOFTWARE REQUIRED:

• Ubuntu (or WSL Ubuntu)

• g++

• CMake

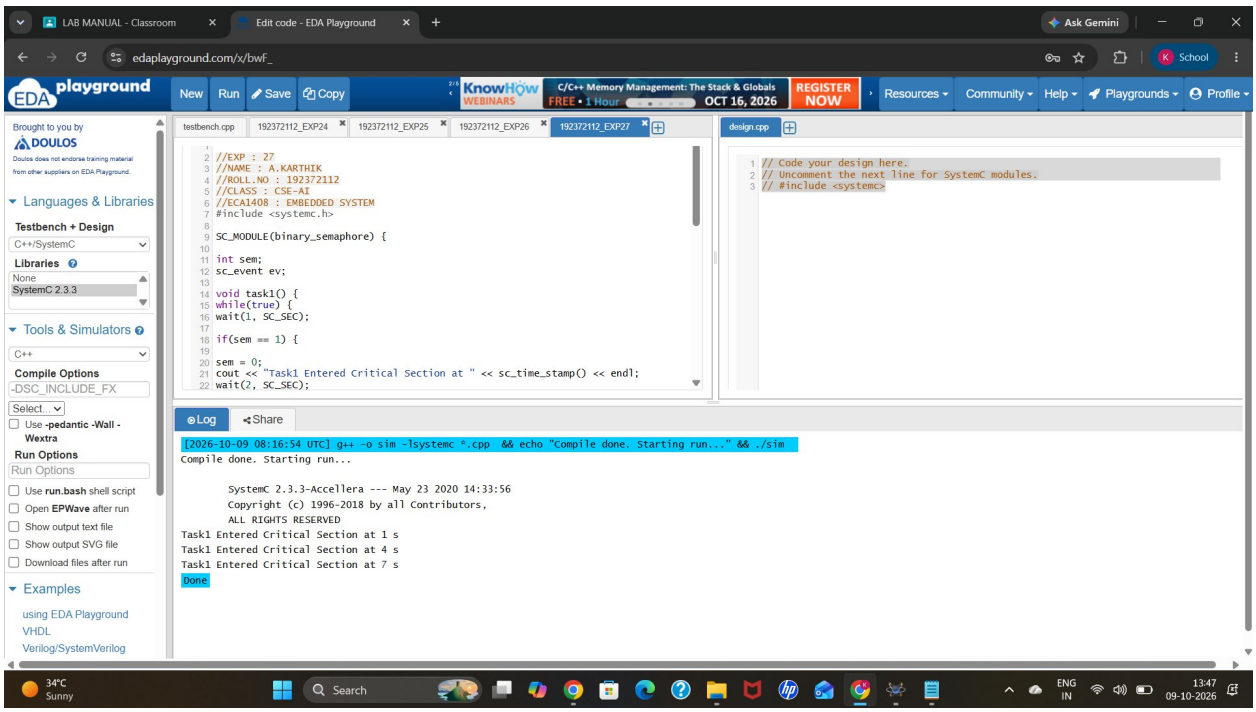
• SystemC

PROGRAM:

```
#include <systemc.h>
SC_MODULE(binary_semaphore) {
int sem;
sc_event ev;
void task1() {
while(true) {
wait(1, SC_SEC);
if(sem == 1) {
sem = 0;
cout << "Task1 Entered Critical Section at " << sc_time_stamp() << endl;
wait(2, SC_SEC);
sem = 1;
ev.notify();
}
else {
wait(ev);
}
}
```

```
}
void task2() {
while(true) {
wait(2, SC_SEC);
if(sem == 1) {
sem = 0;
cout << "Task2 Entered Critical Section at " << sc_time_stamp() << endl;
wait(1, SC_SEC);
sem = 1;
ev.notify();
}
else {
wait(ev);
}
}
}
SC_CTOR(binary_semaphore) {
sem = 1;
SC_THREAD(task1);
SC_THREAD(task2);
}
};
int sc_main(int argc, char* argv[]) {
binary_semaphore obj("Binary_Semaphore");
sc_start(10, SC_SEC);
return 0;
}
```

OUTPUT:



RESULT:

Thus the Binary Semaphore was created using SystemC and verified.